

SCJP 1.6 (CX-310-065 , CX-310-066)

Subject: Collections and Generics

Collections , hashCode, equals, generics , list, map, set , navigableMap, NavigableSet

Total Questions : 77

Prepared by : <http://www.javacertifications.net>

SCJP 6.0: Collections and Generics

Questions Question - 1

What is the output?

```
import java.util.ArrayList;
import java.util.List;
import java.util.NavigableSet;
import java.util.TreeSet;

public class Test {
    public static void main(String... args) {

        List<Integer> lst = new ArrayList<Integer>();
        lst.add(34);
        lst.add(6);
        lst.add(2);
        lst.add(8);
        lst.add(7);
        lst.add(10);

        NavigableSet<Integer> nvset = new TreeSet(lst);
        System.out.println(nvset.lower(6)+" "+nvset.higher(6)+ "
"+ nvset.lower(2));
    }
}
```

1.1 2 7 10 34 null

2.2 7 null

3.2 7 34

4.1 2 7 10 34

Explanation :

B is the correct answer.

□lower() Returns the greatest element in this set strictly less than the given element, or null if there is no such element.

□higher() Returns the least element in this set strictly greater than the given element, or null if there is no such element.

Question - 2

What is the output?

```
import java.util.ArrayList;
```

```

import java.util.Iterator;
import java.util.List;
import java.util.NavigableSet;
import java.util.TreeSet;

public class Test {
    public static void main(String... args) {

        List<Integer> lst = new ArrayList<Integer>();
        lst.add(34);
        lst.add(6);
        lst.add(2);
        lst.add(8);
        lst.add(7);
        lst.add(10);

        NavigableSet<Integer> nvset = new TreeSet(lst);
        Iterator<Integer> iterator =
(Iterator)nvset.descendingIterator();
        while (iterator.hasNext()){
            System.out.print(iterator.next() + " ");
        }

    }
}

```

- 1.Compile error : No method name like descendingIterator()
- 2.34 10 8 7 6 2
- 3.2 6 7 8 10 34
- 4.None of the above

Explanation :

B is the correct answer.

☐descendingIterator() Returns the elements in descending order.

Question - 3

What is the output?

```

import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;
import java.util.NavigableSet;
import java.util.TreeSet;

public class Test {
    public static void main(String... args) {

        List<Integer> lst = new ArrayList<Integer>();
        lst.add(34);
        lst.add(6);
        lst.add(2);
        lst.add(8);
        lst.add(7);

```

```

        lst.add(10);

        NavigableSet<Integer> nvset = new TreeSet(lst);
        Iterator<Integer> iterator = (Iterator)nvset.iterator();
        while (iterator.hasNext()){
            System.out.print(iterator.next() + " ");
        }

    }
}

```

1.Compile error : No method name like iterator()

2.34 10 8 7 6 2

3.2 6 7 8 10 34

4.None of the above

Explanation :

C is the correct answer.

S iterator() Returns the elements in ascending order.

Question - 4

Fill the gap.

```

public interface NavigableSet<E> extends <E> {
}

```

Use each fragment at most once

SortedSet

Set

AbstractSet

SortedSet

Question - 5

What is the output?

```

import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;
import java.util.NavigableSet;
import java.util.TreeSet;

public class Test {
    public static void main(String... args) {

        List<Integer> lst = new ArrayList<Integer>();
        lst.add(34);
        lst.add(6);
        lst.add(2);
        lst.add(8);
        lst.add(7);
        lst.add(10);
    }
}

```

```

        NavigableSet<Integer> nvset = new TreeSet(lst);
        System.out.println(nvset.headSet(10));

    }
}

```

1. Compile error : No method name like headSet()
2. 2, 6, 7, 8, 10
3. 2, 6, 7, 8
4. None of the above

Explanation :

C is the correct answer.

□ headSet(10) Returns the elements elements are strictly less than 10.

K headSet(10,false) Returns the elements elements are strictly less than 10.

; headSet(10,true) Returns the elements elements are strictly less than or equal to 10.

Question - 6

What is the output?

```

import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;
import java.util.NavigableSet;
import java.util.TreeSet;

public class Test {
    public static void main(String... args) {

        List<Integer> lst = new ArrayList<Integer>();
        lst.add(34);
        lst.add(6);
        lst.add(2);
        lst.add(8);
        lst.add(7);
        lst.add(10);

        NavigableSet<Integer> nvset = new TreeSet(lst);
        System.out.println(nvset.headSet(10,true));

    }
}

```

1. Compile error : No method name like headSet()
2. 2, 6, 7, 8, 10
3. 2, 6, 7, 8
4. None of the above

Explanation :

B is the correct answer.

headSet(10) Returns the elements elements are strictly less than 10.

K headSet(10,false) Returns the elements elements are strictly less than 10.

; headSet(10,true) Returns the elements elements are strictly less than or equal to 10.

Question - 7

What is the output?

```
import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;
import java.util.NavigableSet;
import java.util.TreeSet;

public class Test {
    public static void main(String... args) {

        List<Integer> lst = new ArrayList<Integer>();
        lst.add(34);
        lst.add(6);

        NavigableSet<Integer> nvset = new TreeSet(lst);
        System.out.println(nvset.pollFirst() + " " +
nvset.pollFirst() + " " + nvset.pollLast());

    }
}
```

1.Compile error : No method name like pollFirst() or pollLast()

2.6 34 null

3.6 34 34

4.6 34 6

Explanation :

B is the correct answer.

U pollFirst() Retrieves and removes the first (lowest) element, or returns null if this set is empty.

U pollLast() Retrieves and removes the last (highest) element, or returns null if this set is empty.

Question - 8

What is the output?

```
import java.util.ArrayList;
import java.util.List;
import java.util.NavigableSet;
import java.util.TreeSet;
```

```

public class Test {
    public static void main(String... args) {

        List<Integer> lst = new ArrayList<Integer>();
        lst.add(34);
        lst.add(6);
        lst.add(6);
        lst.add(6);
        lst.add(6);
        lst.add(6);

        NavigableSet<Integer> nvset = new TreeSet(lst);
        nvset.pollFirst();
        nvset.pollLast();
        System.out.println(nvset.size());

    }
}

```

1.Compile error : No method name like pollFirst() or pollLast()

2.0

3.2

4.5

Explanation :

B is the correct answer.

U pollFirst() Retrieves and removes the first (lowest) element, or returns null if this set is empty.

U pollLast() Retrieves and removes the last (highest) element, or returns null if this set is empty.

Question - 9

What is the output?

```

public class Test {
    public static void main(String... args) {

        List<Integer> lst = new ArrayList<Integer>();
        lst.add(34);
        lst.add(6);
        lst.add(6);
        lst.add(6);
        lst.add(6);
        lst.add(6);
        lst.add(5);

        NavigableSet<Integer> nvset = new TreeSet(lst);
        System.out.println(nvset.tailSet(6));

    }
}

```

1.Compile error : No method name like tailSet()

2.6 34
3.6 34 34
4.6 34 6

Explanation :

B is the correct answer.

❑tailSet(6) Returns elements are greater than or equal to 6.

Question - 10

Fill the gap.

```
public interface NavigableMap<K,V> extends <K,V> {  
}
```

Use each fragment at most once

SortedMap

Map

AbstractMap

SortedMap

Question - 11

What is the output?

```
import java.util.NavigableMap;  
import java.util.concurrent.ConcurrentSkipListMap;
```

```
public class Test {  
    public static void main(String... args) {  
  
        NavigableMap <Integer, String>navMap = new  
        ConcurrentSkipListMap<Integer, String>();  
  
        navMap.put(4, "April");  
        navMap.put(5, "May");  
        navMap.put(6, "June");  
        navMap.put(1, "January");  
        navMap.put(2, "February");  
        navMap.put(3, "March");  
  
        System.out.print(navMap.firstEntry());  
  
    }  
}
```

1.Compile error : No method name like firstEntry()

2.1=January

3.6=June

4.4=April

Explanation :

B is the correct answer.

1 firstEntry() Returns a key-value mapping associated with the least key in this map, or null if the map is empty.

Question - 12

What is the output?

```
import java.util.NavigableMap;
import java.util.concurrent.ConcurrentSkipListMap;

public class Test {
    public static void main(String... args) {

        NavigableMap <Integer, String>navMap = new
        ConcurrentSkipListMap<Integer, String>();

        navMap.put(4, "April");
        navMap.put(5, "May");
        navMap.put(6, "June");
        navMap.put(1, "January");
        navMap.put(2, "February");
        navMap.put(3, "March");

        System.out.print(navMap.lastEntry());
    }
}
```

1.Compile error : No method name like lastEntry()

2.1=January

3.6=June

4.4=April

Explanation :

C is the correct answer.

□lastEntry() Returns a key-value mapping associated with the greatest key in this map, or null if the map is empty.

Question - 13

What is the output?

```
import java.util.NavigableMap;
import java.util.concurrent.ConcurrentSkipListMap;

public class Test {
    public static void main(String... args) {

        NavigableMap <Integer, String>navMap = new
```



```

        ConcurrentSkipListMap<Integer, String>();

        System.out.print(navMap.lastEntry());

    }
}

```

- 1.Compile error : No method name like lastEntry()
- 2.null
- 3.NullPointerException
- 4.None of the above

Explanation :

B is the correct answer.

❑lastEntry() Returns a key-value mapping associated with the greatest key in this map, or null if the map is empty.

Question - 14

What is the output?

```

import java.util.NavigableMap;
import java.util.concurrent.ConcurrentSkipListMap;

public class Test {
    public static void main(String... args) {

        NavigableMap <Integer, String>navMap = new
        ConcurrentSkipListMap<Integer, String>();

        navMap.put(4, "April");
        navMap.put(5, "May");
        navMap.put(6, "June");
        navMap.put(1, "January");
        navMap.put(2, "February");
        navMap.put(3, "March");

        System.out.print(navMap.pollFirstEntry()+ " " +
        navMap.pollLastEntry()+ " "+navMap.pollFirstEntry());

    }
}

```

- 1.Compile error : No method name like pollFirstEntry() or pollLastEntry()
- 2.1=January 6=June 2=February
- 3.NullPointerException
- 4.4=April 3=March 5=May

Explanation :

B is the correct answer.

- O pollFirstEntry() Removes and returns a key-value mapping associated with the least key in this map, or null if the map is empty.
- K pollLastEntry() Removes and returns a key-value mapping associated with the greatest key in this map, or null if the map is empty.

Question - 15

What is the output?

```
import java.util.NavigableMap;
import java.util.concurrent.ConcurrentSkipListMap;

public class Test {
    public static void main(String... args) {

        NavigableMap <Integer, String>navMap = new
        ConcurrentSkipListMap<Integer, String>();

        navMap.put(4, "April");
        navMap.put(5, "May");
        navMap.put(6, "June");
        navMap.put(1, "January");
        navMap.put(2, "February");
        navMap.put(3, "March");

        navMap.pollFirstEntry();
        navMap.pollLastEntry();
        navMap.pollFirstEntry();
        System.out.println(navMap.size());

    }
}
```

- 1.Compile error : No method name like pollFirstEntry() or pollLastEntry()
- 2.3
- 3.6
- 4.None of the above

Explanation :

B is the correct answer.

* pollFirstEntry() Removes and returns a key-value mapping associated with the least key in this map, or null if the map is empty.

K pollLastEntry() Removes and returns a key-value mapping associated with the greatest key in this map, or null if the map is empty.

Question - 16

Fill the gap

```
public class Test {

    public static void main(String... args) throws Exception {
        <String> lst = new LinkedList<String>();
    }
}
```

```

        lst.add("one");
        lst.add("two");
        System.out.println(lst.poll());
    }
}

```

Use the following fragments zero or many times

Map

Sequence

Queue

Question - 17

Fill the gap

```

public class Test {

    public static void main(String... args) throws Exception {
        LinkedList<String> lst = new LinkedList<String>();
        lst.add("one");
        lst.add("two");
        System.out.println(lst.);
    }
}

```

Use the following fragments zero or many times

getIndex()

addIndex()

poll()

Question - 18

What is the output?

```

public class Test {

    public static void main(String... args) throws Exception {
        Queue<String> lst = new PriorityQueue<String>();
        lst.add("one");
        lst.add("two");
        System.out.println(lst.poll() +" "+ lst.poll()+ " "+
+lst.poll());
    }
}

```

1.one two null]> 3.If the hashCode() comparison == returns true, the equals() method might return true

4.If the hashCode() comparison == returns true, the equals() method must return true

Explanation :

A, B and C is the correct answer.

Two dissimilar objects can return the same hashCode value. if the hashCode() comparison returns ==, the two objects might or might not be equal. But If the equals() method returns true, the hashCode() comparison == must return true.

Question - 20

Fill the gap to get output aa ab ac

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class Test {
    public static void main(String... args) {

        List<String> a = new ArrayList<String>();
        a.add("aa"); a.add("ac"); a.add("ab");

        // insert code

        for(String s1: a) System.out.println(s1);
    }
}
```

Use each fragment at most once

Collections.sort(a);

Collections.reverse(a);

Collections.sort(a);

Question - 21

Fill the gap to get output ac ab aa

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class Test {
    public static void main(String... args) {

        List<String> a = new ArrayList<String>();
```

```

        a.add("aa"); a.add("ac"); a.add("ab");

        // insert code

        for(String s1: a) System.out.println(s1);
    }

}

```

Use each fragment at most once

Collections.sort(a);

Collections.reverse(a);

Collections.sort(a);

Collections.reverse(a);

Question - 22

If you are comparing two instances of the same class, given that the equals() and hashCode() methods have been properly overridden?

- 1.If the equals() method returns true, the hashCode() comparison == must return true
- 2.If the equals() method returns false, the hashCode() comparison == might return true]
- 3.If the hashCode() comparison == returns true, the equals() method might return true
- 4.All of the above

Explanation :

D is the correct answer.

Two dissimilar objects can return the same hashCode value. if the hashCode() comparison returns ==, the two objects might or might not be equal. But If the equals() method returns true, the hashCode() comparison == must return true.

Question - 23

What is the output for the below code?

```

import java.util.LinkedList;
import java.util.Queue;

public class Test {
    public static void main(String... args) {

        Queue<String> q = new LinkedList<String>();
        q.add("newyork");
    }
}

```

```

        q.add("ca");
        q.add("texas");
        show(q);
    }

    public static void show(Queue q) {
        q.add(new Integer(11));
        while (!q.isEmpty())
            System.out.print(q.poll() + " ");
    }
}

```

- 1.Compile error : Integer can't add
- 2.newyork ca texas 11
- 3.newyork ca texas
- 4.None of the above

Explanation :

B is the correct answer.

□ q was originally declared as Queue<String>, But in show() method it is passed as an untyped Queue. nothing in the compiler or JVM prevents us from adding an Integer after that.

@ If the show method signature is public static void show(Queue<String> q) than you can't add Integer, Only String allowed. But public static void show(Queue q) is untyped Queue so you can add Integer.

U poll() Retrieves and removes the head of this queue, or returns null if this queue is empty.

Question - 24

What is the output?

```

import java.util.LinkedList;
import java.util.Queue;

public class Test {
    public static void main(String... args) {

        Queue<String> q = new LinkedList<String>();
        q.add("newyork");
        q.add("ca");
        q.add("texas");
        show(q);
    }

    public static void show(Queue<String> q) {
        q.add(new Integer(11));
        while (!q.isEmpty())
            System.out.print(q.poll() + " ");
    }
}

```

```
}
```

- 1.Compile error : Integer can't add
- 2.newyork ca texas 11
- 3.newyork ca texas
- 4.None of the above

Explanation :

A is the correct answer.

@ method signature is public static void show(Queue<String> q) so you can't add Integer, Only String allowed. But if method signature is public static void show(Queue q) then it is untyped Queue you can add Integer.

U poll() Retrieves and removes the head of this queue, or returns null if this queue is empty.

Question - 25

What is the output for the below code?

```
import java.util.Iterator;
import java.util.Set;
import java.util.TreeSet;

public class Test {
    public static void main(String... args) {

        Set s = new TreeSet();
        s.add("7");
        s.add(9);
        Iterator itr = s.iterator();
        while (itr.hasNext())
            System.out.print(itr.next() + " ");

    }
}
```

- 1.Compile error
- 2.Runtime Exception
- 3.7 9
- 4.None of the above

Explanation :

B is the correct answer.

□Without generics, the compiler does not know what type is appropriate for this TreeSet, so it allows everything to compile. But at runtime the TreeSet will try to sort the elements as they are added, and when it tries to compare an Integer with a String it will throw a ClassCastException.

| Exception in thread "main" java.lang.ClassCastException: java.lang.String cannot be cast to java.lang.Integer.

Question - 26

Which collection class grows or shrinks its size and provides indexed access to its elements, but methods are not synchronized?

- 1.java.util.ArrayList
- 2.java.util.List
- 3.java.util.HashSet
- 4.java.util.Vector

Explanation :

A is the correct answer.

□ArrayList provides an index to its elements and methods are not synchronized.

Question - 27

What is the output?

```
import java.util.Iterator;
import java.util.TreeSet;

public class Test {
    public static void main(String... args) {

        TreeSet s1 = new TreeSet();
        s1.add("one");
        s1.add("two");
        s1.add("three");
        s1.add("one");

        Iterator it = s1.iterator();
        while (it.hasNext() ) {
            System.out.print( it.next() + " " );
        }

    }
}
```

- 1.one three two
- 2.Runtime Exception
- 3.one three two one
- 4.one two three

Explanation :

A is the correct answer.

; TreeSet assures no duplicate entries.it will return elements in natural order, which, for Strings means alphabetical.

Question - 28

Which of the following methods are members of the Vector class and allow you to input a new element

- 1.addElement
- 2.insert
- 3.append
- 4.addItem

Explanation :

A is the correct answer.

No Explanation Available

Question - 29

Which of the following statements are true?

- 1.At the root of the collection hierarchy is a class called Collection
- 2.The collection interface contains a method called enumerator
- 3.The iterator method returns an instance of the Vector class
- 4.The Set interface is designed for unique elements

Explanation :

D is the correct answer.

Collection is an interface, not a class. The Collection interface includes a method called iterator. This returns an instance of the Iterator class which has some similarities with Enumerators. The name set should give away the purpose of the Set interface as it is analogous to the Set concept in relational databases which implies uniqueness.

Question - 30

How does the set collection deal with duplicate elements?

- 1.An exception is thrown if you attempt to add an element with a duplicate value
- 2.The add method returns false if you attempt to add an element with a duplicate value
- 3.A set may contain elements that return duplicate values from a call to the equals method
- 4.Duplicate values will cause an error at compile time

Explanation :

B is the correct answer.

The add method returns false if you attempt to add an element with a duplicate value

Question - 31

Which most closely matches a description of a Java Map?

- 1.A vector of arrays for a 2D geographic representation
- 2.A class for containing unique array elements
- 3.A class for containing unique vector elements
- 4.An interface that ensures that implementing classes cannot contain duplicate keys

Explanation :

D is the correct answer.

Map does not allow duplicate key. It override the the value.

Question - 32

Is Map interface extends Collection interface ?

- 1.true
- 2.false
- 3.can't say
- 4.None of the above

Explanation :

B is the correct answer.

```
public interface Map { }
```

Question - 33

```
List<String> list = new ArrayList<String>
```

In the above list, is it only can contain String objects ?

- 1.yes, it only can contain String objects
- 2.No, it can contain any objects
- 3.None of the above
- 4.None of the above

Explanation :

A is the correct answer.

List list = new ArrayList In the above list, it only can contain String objects

Question - 34

Which statement is true ?

- 1.TreeSet is slower than HashSet.
- 2.HashSet is slower than TreeSet .
- 3.Both has the same performance.
- 4.None of the above

Explanation :

A is the correct answer.

TreeSet, which stores its elements in a red-black tree, orders its elements based on their values; it is substantially slower than HashSet

Question - 35

What is true statement about SortedMap ?

- 1.A map that further guarantees that it will be in ascending key order
- 2.A map that further guarantees that it will be in ascending value order
- 3.None of the above
- 4.None of the above

Explanation :

A is the correct answer.

A map that further guarantees that it will be in ascending key order, sorted according to the natural ordering of its keys, or by a comparator provided at sorted map creation time. All keys inserted into a sorted map MUST implement the Comparable interface (or be accepted by the specified comparator).

Question - 36

What is the output of the below code ?

```
public class NameBean {
    private String str;

    NameBean(String str ){
        this.str = str;
    }

    public String toString() {
        return str;
    }
}
```

```

import java.util.HashSet;

public class CollClient {

    public static void main(String ... sss) {
        HashSet myMap = new HashSet();
        String s1 = new String("das");
        String s2 = new String("das");
        NameBean s3 = new NameBean("abcdef");
        NameBean s4 = new NameBean("abcdef");

        myMap.add(s1);
        myMap.add(s2);
        myMap.add(s3);
        myMap.add(s4);

        System.out.println(myMap);
    }
}

```

- 1.das abcdef abcdef
- 2.das das abcdef abcdef
- 3.das abcdef
- 4.None Of the above

Explanation :

A is the correct answer.

you have to implements 'equals' and 'hashCode' methods to get unique Set for user defined objects.

Question - 37

What is the output of bellow code ?

```

public class Bean{
    private String str;

    Bean(String str ){
        this.str = str;
    }

    public String getStr() {
        return str;
    }

    public boolean equals(Object o){
        if (!(o instanceof Bean)) {
            return false;
        }

        return ((Bean) o).getStr().equals(str);
    }

    public int hashCode() {

```

```

        return 12345;
    }

    public String toString() {
        return str;
    }
}

import java.util.HashSet;

public class CollClientFixed {

    public static void main(String ... sss) {
        HashSet myMap = new HashSet();
        String s1 = new String("das");
        String s2 = new String("das");
        Bean s3 = new Bean("abcdef");
        Bean s4 = new Bean("abcdef");

        myMap.add(s1);
        myMap.add(s2);
        myMap.add(s3);
        myMap.add(s4);

        System.out.println(myMap);
    }
}

```

- 1.das abcdef
- 2.das abcdef das abcdef
- 3.das das abcdef abcdef
- 4.None of the above

Explanation :

A is the correct answer.

implements 'equals' and 'hashCode' methods to get unique result in Set.

Question - 38

What is the output for the bellow code ?

```

public class Bean {
    private String str;

    Bean(String str ){
        this.str = str;
    }

}

public class Test {

```

```

public static void main(String argv[]){

    TreeSet myMap = new TreeSet();
    String s1 = new String("abcdef");
    String s2 = new String("abcdef");
    Bean s3 = new Bean("abcdef");
    Bean s4 = new Bean("abcdef");

    myMap.add(s1);
    myMap.add(s2);
    myMap.add(s3);
    myMap.add(s4);

    System.out.println(myMap);

}
}

```

- 1.abcdef abcdef abcdef abcdef
- 2.abcdef abcdef abcdef
- 3.java.lang.ClassCastException
- 4.None of the above

Explanation :

C is the correct answer.

Every Object need to implements comparable interface to sort the objects in TreeSet .

Question - 39

What is the true statement about TreeSet ?

- 1.This class guarantees that the sorted set will be in ascending element order
- 2.All the added Objects in TreeSet need to implements Comparable interface
- 3.Both are true
- 4.None of the above

Explanation :

C is the correct answer.

This class guarantees that the sorted set will be in ascending element order, All the added Objects in TreeSet need to implements Comparable interface

Question - 40

Which statement is true ?

1. LinkedList - Doubly-linked list implementation of the List interface
2. LinkedList - provide better performance than the ArrayList implementation if elements are frequently inserted or deleted within the list
3. Both are true
4. None of the above

Explanation :

C is the correct answer.

LinkedList implements List, Queue - Doubly-linked list implementation of the List interface. May provide better performance than the ArrayList implementation if elements are frequently inserted or deleted within the list. Can be used as a double-ended queue (deque). Also implements the Queue interface. When accessed via the Queue interface, LinkedList behaves as a FIFO queue

Question - 41

Synchronized ~~resizable~~-array implementation of the **List** interface is _____ ?

1. Vector
2. Hashtable
3. ArrayList
4. None of the above

Explanation :

A is the correct answer.

Vector implements List, RandomAccess - Synchronized resizable-array implementation of the List interface with additional "legacy methods."

Question - 42

Synchronized hash table implementation of the **Map** interface that DOES NOT allow null keys or values is _____ ?

1. Hashtable
2. Vector
3. HashMap
4. None of the above

Explanation :

A is the correct answer.

Hashtable implements Map - Synchronized hash table implementation of the Map interface that DOES NOT allow null keys or values, with additional "legacy

methods." To successfully store and retrieve objects from a hashtable, the objects used as keys must implement the hashCode method and the equals method. Unlike the new collection implementations, Hashtable is synchronized

Question - 43

What is the output for the below code?

```
public class Test {

    public static void main(String... args) throws Exception {
        Queue<String> lst = new PriorityQueue<String>();
        lst.add("one");
        lst.add("two");
        System.out.println(lst.poll() + " " + lst.poll() + " "
+lst.poll());
    }
}
```

- 1.one two null
- 2.one one one
- 3.one two one
- 4.one two two

Explanation :

A is the correct answer.

poll() returns the highest priority element, AND removes it from the queue. and return null if queue is empty.

Question - 44

What is the output of the bellow code ?

```
public class Test {

    public static void main(String argv[]){

        ArrayList list = new ArrayList();
        ArrayList<String> listStr = list;
        ArrayList<StringBuffer> listBuf = list;
        listStr.add(0, "Hello");
        StringBuffer buff = listBuf.get(0);
        System.out.println(buff.toString());
    }
}
```

- 1.Hello

- 2.Compile time error
- 3.java.lang.ClassCastException
- 4.None of the above

Explanation :

C is the correct answer.

java.lang.ClassCastException to cast String to StringBuffer.

Question - 45

What is the output for the bellow code ?

```
import java.util.*;

public class FindDups {
    public static void main(String[] args) {
        Set<String> s = new HashSet<String>();
        for (String a : args)
            if (!s.add(a))
                System.out.println("Duplicate detected: " + a);

        System.out.println(s.size() + " distinct words: " + s);
    }
}
```

when run using command line using command

```
java FindDups i came i saw i dir
```

- 1.Duplicate detected: i Duplicate detected: i 4 distinct words: [i, left, saw, dir]
- 2.Duplicate detected: i 4 distinct words: [i, left, saw, came]
- 3.4 distinct words: [i, i, i ,left, saw, came]
- 4.None of the above

Explanation :

A is the correct answer.

add() method of Set turn boolean. If there is any existing element in the HashSet then return false.

Question - 46

Which statement is true?

- 1.interface List extends Collection
- 2.Class List extends Collection
- 3.interface List extends Map
- 4.None of the above

Explanation :

A is the correct answer.

The List interface follows. `public interface List extends Collection { }`

Question - 47

`Is it true ?`

`"Lists may contain duplicate elements"`

- 1.true
- 2.false
- 3.None of the above
- 4.None of the above

Explanation :

A is the correct answer.

A List is an ordered Collection (sometimes called a sequence). Lists may contain duplicate elements

Question - 48

`java.util.Collections` is class or interface ?

- 1.class
- 2.interface
- 3.both
- 4.None of the above

Explanation :

A is the correct answer.

`public class Collections { }`

Question - 49

`In the bellow code ,`

`public class Test {`

`public static void main(String argv[]){`

`ArrayList alist = new ArrayList();
alist.add("sa");`

```

        alist.add("ga");
        alist.add("aa");
        alist.add("da");
        // Sort ArrayList code
    }
}

```

How to sort the ArrayList ?

- 1.Collections.sort(alist);
- 2.Collection.sort(alist);
- 3.alist.sort();
- 4.None of the above

Explanation :

A is the correct answer.

Collections.sort(alist) sort the List ascending order.

Question - 50

In the bellow code ,

```

public class Test {

    public static void main(String argv[]){

        ArrayList alist = new ArrayList();
        alist.add("sa");
        alist.add("ga");
        alist.add("aa");
        alist.add("da");
        // Suffle ArrayList code
    }
}

```

How to Suffle the ArrayList ?

- 1.Collections.shuffle(alist);
- 2.Collection.shuffle(alist);
- 3.alist.shuffle();
- 4.None of the above

Explanation :

A is the correct answer.

Collections.shuffle(alist) , suffle the List.

Question - 51

How can we create thread safe List ?

- 1.Collections.synchronizedList(List list);
- 2.Collection.synchronizedList(List list);
- 3.List.synchronizedList();
- 4.None of the above

Explanation :

A is the correct answer.

Collections.synchronizedList(List list) returns Thread safe List.

Question - 52

What is the output ?

```
public class Test {  
  
    public static void main(String argv[]) throws  
    InterruptedException{  
  
        int time = Integer.parseInt("5");  
        Queue queue = new LinkedList();  
        for (int i = time; i >= 0; i--)  
            queue.add(new Integer(i));  
        while (!queue.isEmpty()) {  
            System.out.println(queue.remove());  
            Thread.sleep(1000);  
        }  
  
    }  
}
```

- 1.5 4 3 2 1 0
- 2.Comiple time error , Queue not valid Java Interface
- 3.Runtime Exception
- 4.None of the above

Explanation :

A is the correct answer.

public interface Queue extends Collection

Question - 53

Which statement is true ?

- 1.A priority queue does not permit null elements
- 2.A priority queue does not permit insertion of non-comparable objects
- 3.A priority queue orders elements according to an order specified at construction time
- 4.All of the above

Explanation :

D is the correct answer.

A priority queue does not permit null elements. A priority queue does not permit insertion of non-comparable objects. A priority queue orders elements according to an order specified at construction time

Question - 54

What is the Difference between remove() and poll() method of Queue?

- 1.remove() differs from poll() only in that it throws an exception if this queue is empty.
- 2.remove() differs from poll() only in that it null if this queue is empty.
- 3.Both are true
- 4.None of the above

Explanation :

A is the correct answer.

remove(): Retrieves and removes the head of this queue. This method differs from poll only in that it throws an exception if this queue is empty. poll(): Retrieves and removes the head of this queue, or returns null if this queue is empty.

Question - 55

A collection designed for holding elements prior to processing
_____?

- 1.Queue
- 2.ArrayList
- 3.Map
- 4.Vector

Explanation :

A is the correct answer.

Queue : A collection designed for holding elements prior to processing. Besides basic Collection operations, queues provide additional insertion, extraction, and inspection operations.

Question - 56

What is true about Deque?

- 1.A linear collection that supports element insertion and removal at both ends
- 2.A linear collection that supports element insertion and removal at one end
- 3.The name deque is short for "double ended queue"
- 4.1 and 3

Explanation :

D is the correct answer.

The name deque is short for "double ended queue" .A linear collection that supports element insertion and removal at both ends

Question - 57

What is true about Map ?

- 1.A Map is an object that maps keys to values
- 2.A map cannot contain duplicate keys
- 3.Both are true
- 4.None of the above

Explanation :

C is the correct answer.

A Map is an object that maps keys to values. A map cannot contain duplicate keys:

Question - 58

What is the output of the bellow code, if we run using command line

```
import java.util.*;
```

```
public class Freq {  
    public static void main(String[] args) {  
        Map<String, Integer> m = new HashMap<String, Integer>();  
    }  
}
```

```

        for (String a : args) {
            Integer freq = m.get(a);
            m.put(a, (freq == null) ? 1 : freq + 1);
        }

        System.out.println(m.size() + " distinct words:");
        System.out.println(m);
    }
}

```

command line :
 java Freq if it is to be it is up to me to delegate

- 1.8 distinct words: {to=3, delegate=1, be=1, it=2, up=1, if=1, me=1, is=2}
- 2.7 distinct words: {delegate=1, be=1, it=2, up=1, if=1, me=1, is=2}
- 3.6 distinct words: {to=3, delegate=1, be=1, it=2, up=1, is=2}
- 4.None of the above

Explanation :
 A is the correct answer.

Trace the code your self

Question - 59

What is the output ?

```

public class Test {

    public static void main(String argv[]) {

        //      put value
        Map map = new HashMap();
        map.put("man", "nick");
        map.put("woman", "jane");

        //      retrieve value

        Iterator itr = map.keySet().iterator();
        while(itr.hasNext()){
            String key = (String)itr.next();
            String value = (String)map.get(key);
            System.out.println("key "+key+" value "+value);
        }

    }
}

```

- 1.key man value nick key woman value jane
- 2.Compile time error
- 3.Runtime Exception
- 4.None of the above

Explanation :

A is the correct answer.

Trace the code

Question - 60

HashMap can be **synchronized** by _____ ?

- 1.Map m = Collections.synchronizeMap(hashMap);
- 2.Map m = hashMap.synchronizeMap();
- 3.Map m = Collection.synchronizeMap(hashMap);
- 4.None of the above

Explanation :

A is the correct answer.

HashMap can be synchronized by Map m = Collections.synchronizeMap(hashMap);

Question - 61

HashMap is non **synchronized** whereas **Hashtable** is **synchronized**.
Is the above statement is true ?

- 1.true
- 2.false
- 3.can't say
- 4.None of the above

Explanation :

A is the correct answer.

HashMap is non synchronized whereas Hashtable is synchronized.

Question - 62

HashMap allows null values as key and value whereas **Hashtable** doesn't allow nulls.

Is the above statement is true ?

- 1.true
- 2.false
- 3.None of the above
- 4.None of the above

Explanation :

A is the correct answer.

HashMap allows null values as key and value whereas Hashtable doesn't allow nulls.

Question - 63

Is Iterator fail safe ?

- 1.true
- 2.false
- 3.None of the above
- 4.None of the above

Explanation :

A is the correct answer.

Fail-safe is relevant from the context of iterators. If an iterator has been created on a collection object and some other thread tries to modify the collection object "structurally?", a concurrent modification exception will be thrown. It is possible for other threads though to invoke "set" method since it doesn't modify the collection "structurally?". However, if prior to calling "set", the collection has been modified structurally, "IllegalArgumentException" will be thrown.

Question - 64

"If an iterator has been created on a collection object and some other thread tries to modify the collection object "**structurally**", a concurrent modification exception will be thrown.

Is the above statement is true ?

- 1.true
- 2.false
- 3.can't say
- 4.None of the above

Explanation :

A is the correct answer.

Fail-safe is relevant from the context of iterators. If an iterator has been created on a collection object and some other thread tries to modify the collection object "structurally", a concurrent modification exception will be thrown. It is possible for other threads though to invoke "set" method since it doesn't modify the collection "structurally". However, if prior to calling "set", the collection has been modified structurally, "IllegalArgumentException" will be thrown.

Question - 65

Which of the following interfaces can be used to manage a collection of elements, with no duplication.

1. Vector
2. List
3. Set
4. Map

Explanation :

C is the correct answer.

Lists and Vectors allow duplication (so answers 1 and 2 are incorrect)

Question - 66

Which of the following are valid ways to create a Map collection.

1. Map m = new Map();
2. Map m = new Map(init capacity, increment capacity);
3. Map m = new Map(new Collection());
4. Map is an interface, and cannot be instantiated.

Explanation :

D is the correct answer.

Map is an interface, and cannot be instantiated. You have to do like Map m = new HashMap();

Question - 67

Choose the class that can hold multiple equal objects in an ordered way.

1. List
2. Map
3. Collection

4. None of the above

Explanation :

A is the correct answer.

Refer to the JDK API documentation for further information.

Question - 68

Which one of the following creates an instance of Vector with an initial capacity of 10, and an incremental capacity of 5.

1. new Vector(10, 5);
2. new Vector(5, 10);
3. None. There is no constructor of Vector which provides this feature.
4. Vector is declared as final, and it is therefore not possible to instantiate it.

Explanation :

A is the correct answer.

Constructs an empty vector with the specified initial capacity and capacity increment.

* * @param initialCapacity the initial capacity of the vector. * @param capacityIncrement the amount by which the capacity is * increased when the vector overflows. * @exception IllegalArgumentException if the specified initial capacity is negative public Vector(int initialCapacity, int capacityIncrement) { }

Question - 69

What is the purpose capacityIncrement in the below Vector constructor.

```
public Vector(int initialCapacity, int capacityIncrement) {  
}
```

1. capacityIncrement : the amount by which the capacity is increased when the vector overflows.
2. capacityIncrement : the amount by which the capacity is increased when the vector size is over half of the initialCapacity.
3. None of the above
4. None of the above

Explanation :

A is the correct answer.

capacityIncrement : the amount by which the capacity is increased when the vector overflows.

Question - 70

What is the purpose initialCapacity in the bellow Vector constructor.

```
public Vector(int initialCapacity, int capacityIncrement) {  
}
```

- 1.initial capacity of the vector
- 2.Vector max size
- 3.None of the above
- 4.None of the above

Explanation :

A is the correct answer.

initial capacity of the vector

Question - 71

If we do

```
Vector v = new Vector();
```

what is the initial capacity of the Vector v ?

- 1.15
- 2.10
- 3.8
- 4.11

Explanation :

B is the correct answer.

Vector constructor code is public Vector() { this(10); }

Question - 72

If we do

```
ArrayList lst = new ArrayList();
```

What is the initial capacity of the ArrayList lst ?

- 1.10
- 2.8
- 3.15
- 4.12

Explanation :

A is the correct answer.

```
/** * Constructs an empty list with an initial capacity of ten. */ public ArrayList()  
{ this(10); }
```

Question - 73

If we do

```
Set st = new HashSet(Collection c);
```

Which bellow statement is true ?

- 1.NullPointerException if the specified collection is null
- 2.Create HashSet with empty data if the specified collection is null
- 3.None of the above
- 4.None of the above

Explanation :

A is the correct answer.

NullPointerException if the specified collection is null

Question - 74

What is the default load factor of HashSet ?

- 1..50
- 2..75
- 3.1.5
- 4.None of the above

Explanation :

B is the correct answer.

```
/** * Constructs a new, empty set; the backing
```

Question - 75

Which statement is true ?

- 1.Use ListIterator than Iterator and Enumeration for List types
- 2.Use ArrayList with proper initialization if you don't want thread safe for the collection whenever you add/remove/access objects at end and middle of collection.

3. Use Vector with proper initialization if you want thread safe for the collection whenever you add/remove/access objects at end and middle of collection.
4. All of the above

Explanation :

D is the correct answer.

Lists: 1. Use ArrayList with proper initialization if you don't want thread safe for the collection whenever you add/remove/access objects at end and middle of collection. 2. Use Vector with proper initialization if you want thread safe for the collection whenever you add/remove/access objects at end and middle of collection. 3. Use LinkedList if you don't want thread safe for the collection whenever you add/remove/access objects at beginning of collection. 4. Use synchronized LinkedList if you want thread safe for the collection whenever you add/remove/access objects at beginning of collection. 5. Use ListIterator than Iterator and Enumeration for List types

Question - 76

Which statement is true ?

1. Use HashSet for maintaining unique objects if you don't want thread safe for the collection for all basic(add/remove/access) operations otherwise use synchronized HashSet for thread safe.
2. Use TreeSet for ordered and sorted set of unique objects for non-thread safe collection otherwise use synchronized TreeSet for thread safe
3. All of the above
4. None of the above

Explanation :

C is the correct answer.

1. Use HashSet for maintaining unique objects if you don't want thread safe for the collection for all basic(add/remove/access) operations otherwise use synchronized HashSet for thread safe. 2. Use TreeSet for ordered and sorted set of unique objects for non-thread safe collection otherwise use synchronized TreeSet for thread safe

Question - 77

Which statement is true ?

1. Use HashMap for non-thread safe map collection otherwise use Hashtable for thread safe collection.
2. Use TreeMap for non-thread safe ordered map collection otherwise use synchronized TreeMap for thread safe.
3. All of the above
4. None of the above

Explanation :

C is the correct answer.

1. Use HashMap for non-thread safe map collection otherwise use Hashtable for thread safe collection. 2. Use TreeMap for non-thread safe ordered map collection otherwise use synchronized TreeMap for thread safe.